

分布式面试题

 面试鸭 mianshiya.com

程序员面试 刷题神器

八股文一网打尽 面试从未如此轻松

- ✓ 9000+ 中大厂高频面试题
- ✓ 大厂面试官团队原创优质题解
- ✓ 网页版+小程序+IDE、VSCode插件
- ✓ 真题命中率高，持续更新

微信扫一扫 >>

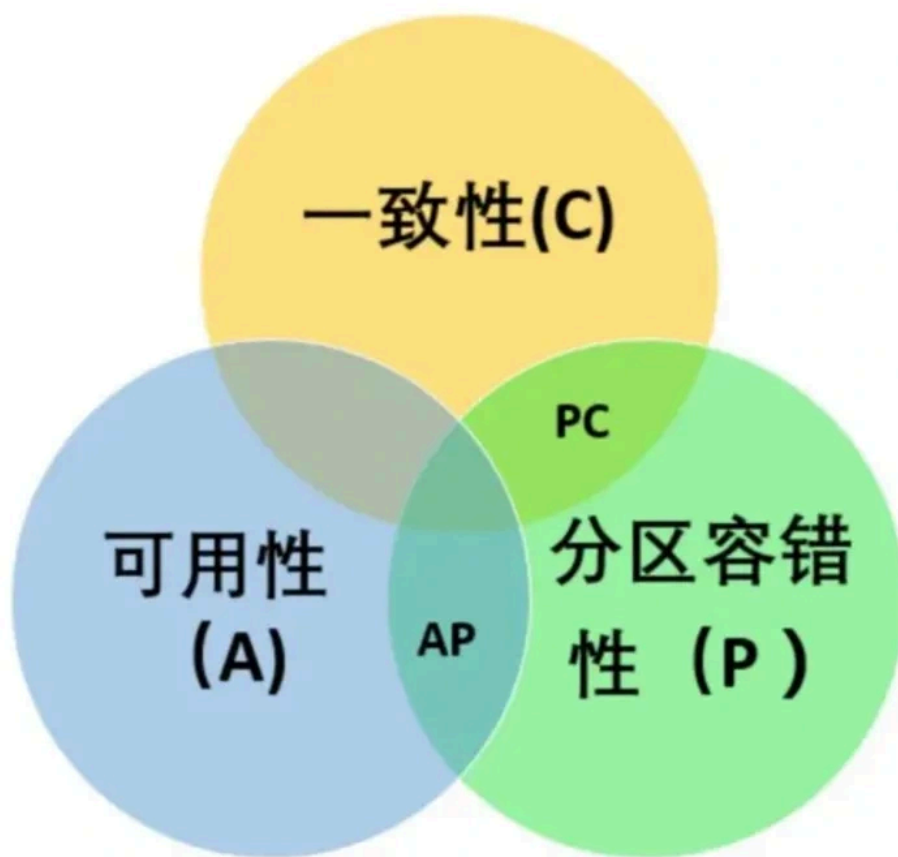
刷高频题拿高薪offer



分布式理论

介绍一下cap理论

CAP 原则又称 CAP 定理, 指的是在一个分布式系统中, Consistency (一致性)、Availability (可用性)、Partition tolerance (分区容错性), **三者不可得兼**

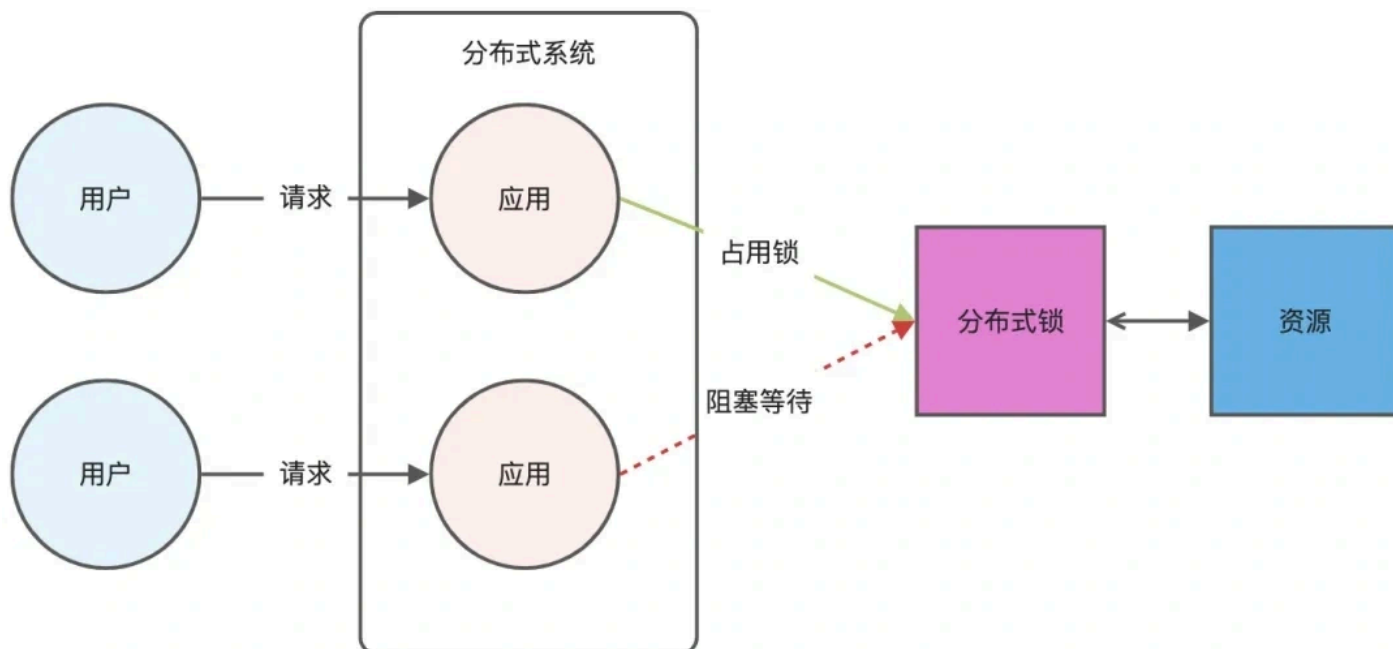


- **一致性(C)** : 在分布式系统中的所有数据备份, 在同一时刻是否同样的值(等同于所有节点访问同一份最新的数据副本)
- **可用性(A)**: 在集群中一部分节点故障后, 集群整体是否还能响应客户端的读写请求(对数据更新具备高可用性)
- **分区容忍性(P)**: 以实际效果而言, 分区相当于对通信的时限要求. 系统如果不能在时限内达成数据一致性, 就意味着发生了分区的情况, 必须就当前操作在 C 和 A 之间做出选择

分布式锁

用redis怎么实现分布式?

分布式锁是用于分布式环境下并发控制的一种机制, 用于控制某个资源在同一时刻只能被一个应用所使用。如下图所示:



Redis 本身可以被多个客户端共享访问，正好就是一个共享存储系统，可以用来保存分布式锁，而且 Redis 的读写性能高，可以应对高并发的锁操作场景。Redis 的 SET 命令有个 NX 参数可以实现「key 不存在才插入」，所以可以用它来实现分布式锁：

- 如果 key 不存在，则显示插入成功，可以用来表示加锁成功；
- 如果 key 存在，则会显示插入失败，可以用来表示加锁失败。

基于 Redis 节点实现分布式锁时，对于加锁操作，我们需要满足三个条件。

- 加锁包括了读取锁变量、检查锁变量值和设置锁变量值三个操作，但需要以原子操作的方式完成，所以，我们使用 SET 命令带上 NX 选项来实现加锁；
- 锁变量需要设置过期时间，以免客户端拿到锁后发生异常，导致锁一直无法释放，所以，我们在 SET 命令执行时加上 EX/PX 选项，设置其过期时间；
- 锁变量的值需要能区分来自不同客户端的加锁操作，以免在释放锁时，出现误释放操作，所以，我们使用 SET 命令设置锁变量值时，每个客户端设置的值是一个唯一值，用于标识客户端；

满足这三个条件的分布式命令如下：

```
SET lock_key unique_value NX PX 10000
```

- lock_key 就是 key 键；
- unique_value 是客户端生成的唯一的标识，区分来自不同客户端的锁操作；
- NX 代表只在 lock_key 不存在时，才对 lock_key 进行设置操作；
- PX 10000 表示设置 lock_key 的过期时间为 10s，这是为了避免客户端发生异常而无法释放锁。

而解锁的过程就是将 lock_key 键删除（del lock_key），但不能乱删，要保证执行操作的客户端就是加锁的客户端。所以，解锁的时候，我们要先判断锁的 unique_value 是否为加锁客户端，是的话，才将 lock_key 键删除。

可以看到，解锁是有两个操作，这时就需要 Lua 脚本来保证解锁的原子性，因为 Redis 在执行 Lua 脚本时，可以以原子性的方式执行，保证了锁释放操作的原子性。

```
// 释放锁时，先比较 unique_value 是否相等，避免锁的误释放
if redis.call("get",KEYS[1]) == ARGV[1] then
    return redis.call("del",KEYS[1])
else
    return 0
end
```

这样一来，就通过使用 SET 命令和 Lua 脚本在 Redis 单节点上完成了分布式锁的加锁和解锁。

还有没有其他方法实现分布式锁？

还可以基于zookeeper来实现分布式。

首先我们来了解一下zookeeper的特性，看看它为什么适合做分布式锁，

zookeeper是一个为分布式应用提供一致性服务的软件，它内部是一个分层的文件系统目录树结构，规定统一一个目录下只能有一个唯一文件名。

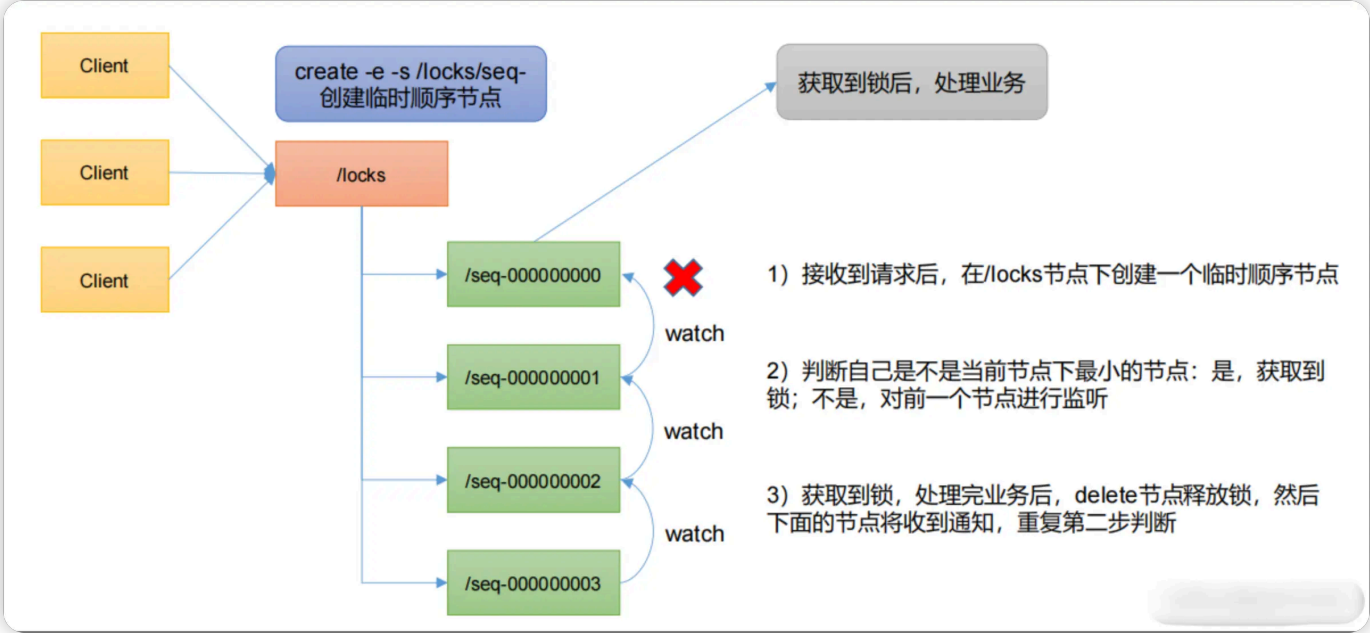
数据模型：

- 永久节点：节点创建后，不会因为会话失效而消失
- 临时节点：与永久节点相反，如果客户端连接失效，则立即删除节点
- 顺序节点：与上述两个节点特性类似，如果指定创建这类节点时，zk会自动在节点名后加一个数字后缀，并且是有序的。

监视器（watcher）：

- 当创建一个节点时，可以注册一个该节点的监视器，当节点状态发生改变时，watch被触发时，ZooKeeper将会向客户端发送且仅发送一条通知，因为watch只能被触发一次。

利用Zookeeper的临时顺序节点和监听机制两大特性，可以帮助我们实现分布式锁。



- 首先得有一个持久节点 `/locks`，路径服务于某个使用场景，如果有多个使用场景建议路径不同。
- 请求进来时首先在 `/locks` 创建临时有序节点，所有会看到在 `/locks` 下面有 `seq-000000000`, `seq-000000001` 等等节点。
- 然后判断当前创建得节点是不是 `/locks` 路径下面最小的节点，如果是，获取锁，不是，阻塞线程，同时设置监听器，监听前一个节点。
- 获取到锁以后，开始处理业务逻辑，最后`delete`当前节点，表示释放锁。
- 后一个节点就会收到通知，唤起线程，重复上面的判断。

zookeeper 实现的分布式锁是强一致性的，**因为它底层的 ZAB协议(原子广播协议), 天然满足 CP, **但是这也意味着性能的下降, 所以不站在具体数据下看 Redis 和 Zookeeper, 代表着性能和一致性的取舍。

如果项目没有强依赖 ZK, 使用 Redis 就好了, 因为现在 Redis 用途很广, 大部分项目中都引用了 Redis, 没必要对此再引入一个新的组件, 如果业务场景对于 Redis 异步方式的同步数据造成锁丢失无法忍受, 在业务层处理就好了。

分布式事务

分布式事务的解决方案你知道哪些？

方案	一致性	性能	复杂度	适用场景
2PC	强一致性	低	中	传统数据库、XA协议
3PC	强一致性	中低	高	需减少阻塞的强一致场景

方案	一致性	性能	复杂度	适用场景
TCC	最终一致性	高	高	高并发业务（支付、库存）
Saga	最终一致性	中	高	长事务、跨服务流程
消息队列	最终一致性	高	中	事件驱动架构
本地消息表	最终一致性	中	低	异步通知（订单-积分）

- 两阶段提交协议（2PC）：为准备阶段和提交阶段。准备阶段，协调者向参与者发送准备请求，参与者执行事务操作并反馈结果。若所有参与者准备就绪，协调者在提交阶段发送提交请求，参与者执行提交；否则发送回滚请求。实现简单，能保证事务强一致性。存在单点故障，协调者故障会影响事务流程；性能低，多次消息交互增加延迟；资源锁导致资源长时间占用，降低并发性能。适用于对数据一致性要求高、并发度低的场景，如金融系统转账业务。
- 三阶段提交协议（3PC）：在 2PC 基础上，将准备阶段拆分为询问阶段和准备阶段，形成询问、准备和提交三个阶段。询问阶段协调者询问参与者能否执行事务，后续阶段与 2PC 类似。降低参与者阻塞时间，提高并发性能，引入超时机制一定程度解决单点故障问题。无法完全避免数据不一致，极端网络情况下可能出现部分提交部分回滚。用于对并发性能有要求、对数据一致性要求相对较低的场景。
- TCC：将业务操作拆分为 Try、Confirm、Cancel 三个阶段。Try 阶段预留业务资源，Confirm 阶段确认资源完成业务操作，Cancel 阶段在失败时释放资源回滚操作。可根据业务场景定制开发，性能较高，减少资源占用时间。开发成本高，需实现三个方法，要处理异常和补偿逻辑，实现复杂度大。适用于对性能要求高、业务逻辑复杂的场景，如电商系统订单处理、库存管理。
- Saga：将长事务拆分为多个短事务，每个短事务有对应的补偿事务。某个短事务失败，按相反顺序执行补偿事务回滚系统状态。性能较高，短事务可并行执行减少时间，对业务侵入性小，只需实现补偿事务。只能保证最终一致性，部分补偿事务失败可能导致系统状态不一致。适用于业务流程长、对数据一致性要求为最终一致性的场景，如旅游系统订单、航班、酒店预订。
- 可靠消息最终一致性方案：基于消息队列，业务系统执行本地事务时将业务操作封装成消息发至消息队列，下游系统消费消息并执行操作，失败则消息队列重试。实现简单，对业务代码修改小，系统耦合度低，能保证数据最终一致性。消息队列可靠性和性能影响大，可能出现消息丢失或延迟，需处理消息幂等性。适用于对数据一致性要求为最终一致性、系统耦合度低的场景，如电商订单支付、库存扣减。
- 本地消息表：业务与消息存储在同一个数据库，利用本地事务保证一致性，后台任务轮询消息表，通过MQ通知下游服务，下游服务消费成功后确认消息，失败则重试。简单可靠，无外部依赖。消息可能重复消费，需幂等设计。适用场景是异步最终一致性（如订单创建后通知积分服务）。

阿里的seata框架了解过吗？

Seata 是开源分布式事务解决方案，支持多种模式：

- **AT模式**：是 Seata 默认的模式，基于支持本地 ACID 事务的关系型数据库。在 AT 模式下，Seata 会自动生成回滚日志，在业务 SQL 执行前后分别记录数据的快照。当全局事务需要回滚时，根据回滚日志将数据恢复到事务开始前的状态。
- **TCC模式**：需要开发者手动编写 Try、Confirm 和 Cancel 三个方法。Try 方法用于对业务资源进行预留，Confirm 方法用于确认资源并完成业务操作，Cancel 方法用于在业务执行失败时释放预留的资源。
- **SAGA 模式**：将一个长事务拆分为多个短事务，每个短事务都有一个对应的补偿事务。当某个短事务执行失败时，会按照相反的顺序执行之前所有短事务的补偿事务，将系统状态回滚到初始状态。

分布式组件

RPC的概念是什么？

RPC 即远程过程调用，允许程序调用运行在另一台计算机上的程序中的过程或函数，就像调用本地程序中的过程或函数一样，而无需了解底层网络细节。

一个典型的 RPC 调用过程通常包含以下几个步骤：

1. **客户端调用**：客户端程序调用本地的一个“伪函数”（也称为存根，Stub），并传入所需的参数。这个“伪函数”看起来和普通的本地函数一样，但实际上它会负责处理远程调用的相关事宜。
2. **请求发送**：客户端存根将调用信息（包括函数名、参数等）进行序列化，然后通过网络将请求发送到服务器端。
3. **服务器接收与处理**：服务器端接收到请求后，将请求信息进行反序列化，然后找到对应的函数并执行。
4. **结果返回**：服务器端将函数的执行结果进行序列化，通过网络发送回客户端。
5. **客户端接收结果**：客户端接收到服务器返回的结果后，将其反序列化，并将结果返回给调用者。

常见的 RPC 框架：

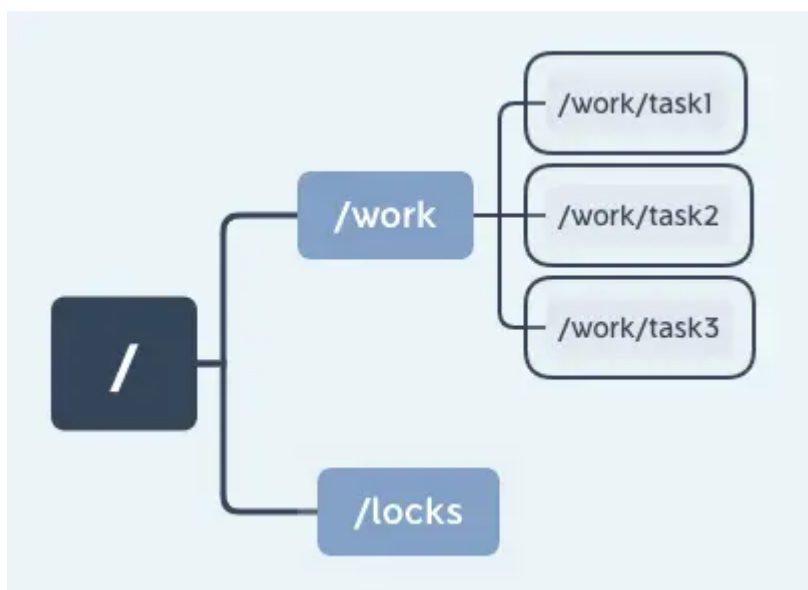
- **gRPC**：由 Google 开发的高性能、开源的 RPC 框架，支持多种编程语言，使用 Protocol Buffers 作为序列化协议，具有高效、灵活等特点。
- **Thrift**：由 Facebook 开发的跨语言的 RPC 框架，支持多种数据传输协议和序列化格式，具有良好的可扩展性和性能。
- **Dubbo**：阿里巴巴开源的高性能 Java RPC 框架，提供了服务治理、集群容错、负载均衡等功能，广泛应用于国内的互联网企业。

zookeeper拿来做什么？核心的原理是什么？

zookeeper是 **分布式协调服务**，它能很好地支持集群部署，并且具有很好的分布式协调能力，可以让我们在分布式部署的应用之间传递数据，保证 **顺序一致性（全序广播）** 而不是 **强一致性**，**以下是其常见的应用场景：

- **配置管理**：在分布式系统中，不同节点往往需要相同的配置信息，如数据库连接参数、服务端口等。ZooKeeper 可以将这些配置信息集中存储，当配置发生变更时，能及时通知到各个节点。例如，一个由多个微服务组成的系统，各个服务实例可以从 ZooKeeper 中获取统一的配置，当配置更新时，ZooKeeper 会通知所有相关服务重新加载配置。
- **服务注册与发现**：服务注册与发现是微服务架构中的关键环节。服务提供者在启动时将自己的服务信息（如服务名称、地址、端口等）注册到 ZooKeeper 中，服务消费者通过 ZooKeeper 查找并获取服务提供者的信息。当服务提供者发生变化（如上线、下线、故障等）时，ZooKeeper 会实时更新服务列表并通知服务消费者。像 Dubbo 框架就可以利用 ZooKeeper 实现服务的注册与发现。
- **分布式锁**：在分布式环境下，多个进程或线程可能会竞争同一资源，为了避免数据不一致等问题，需要实现分布式锁。ZooKeeper 可以通过创建临时顺序节点来实现分布式锁。当一个客户端需要获取锁时，它会在 ZooKeeper 中创建一个临时顺序节点，然后检查自己创建的节点是否是序号最小的节点，如果是，则表示获取到了锁；如果不是，则等待前一个节点释放锁。

ZooKeeper 的数据模型类似于文件系统的树形结构，每个节点称为 Znode。

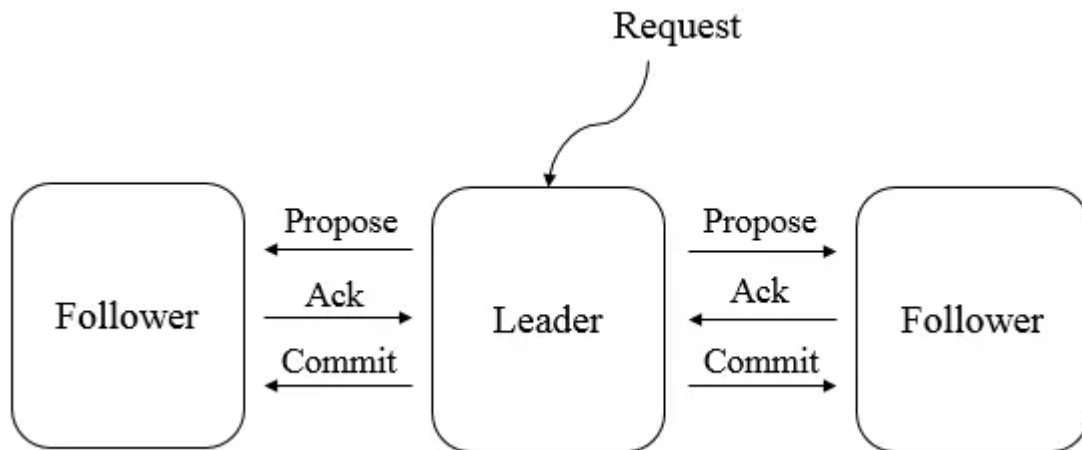


每个 Znode 可以存储数据，也可以有子节点。Znode 有不同的类型，包括持久节点（PERSISTENT）、临时节点（EPHEMERAL）和顺序节点（SEQUENTIAL）。

- **持久节点**：一旦创建，除非主动删除，否则会一直存在。

- **临时节点**：与客户端会话绑定，当客户端会话结束时，临时节点会自动被删除。
- **顺序节点**：在创建时，ZooKeeper 会为其名称添加一个单调递增的序号，保证节点创建的顺序性。

ZooKeeper 使用 ZAB协议来保证集群中数据的一致性。ZAB 协议基于主从架构，有一个领导者（Leader）和多个跟随者（Follower）。



- **消息广播**：当客户端发起写请求时，请求会先到达领导者。领导者将写操作封装成一个事务提案，并广播给所有跟随者。跟随者收到提案后，将其写入本地日志，并向领导者发送确认消息。当领导者收到超过半数跟随者的确认消息后，会发送提交消息给所有跟随者，跟随者收到提交消息后，将事务应用到本地状态机。
- **崩溃恢复**：当领导者出现故障时，ZooKeeper 会进入崩溃恢复阶段。在这个阶段，集群会选举出新的领导者，并确保在新领导者产生之前，不会处理新的写请求。选举过程基于节点的事务 ID 和节点 ID 等信息，保证新选举出的领导者包含了所有已提交的事务。

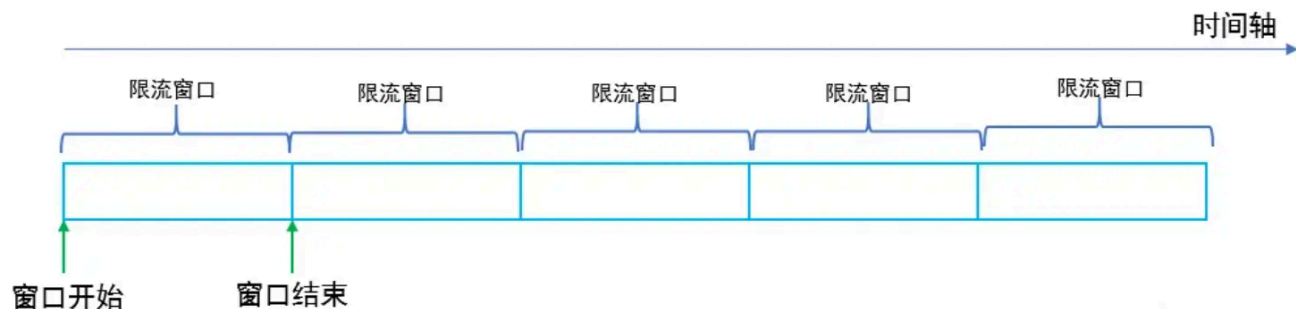
分布式场景

常见的限流算法你知道哪些？？

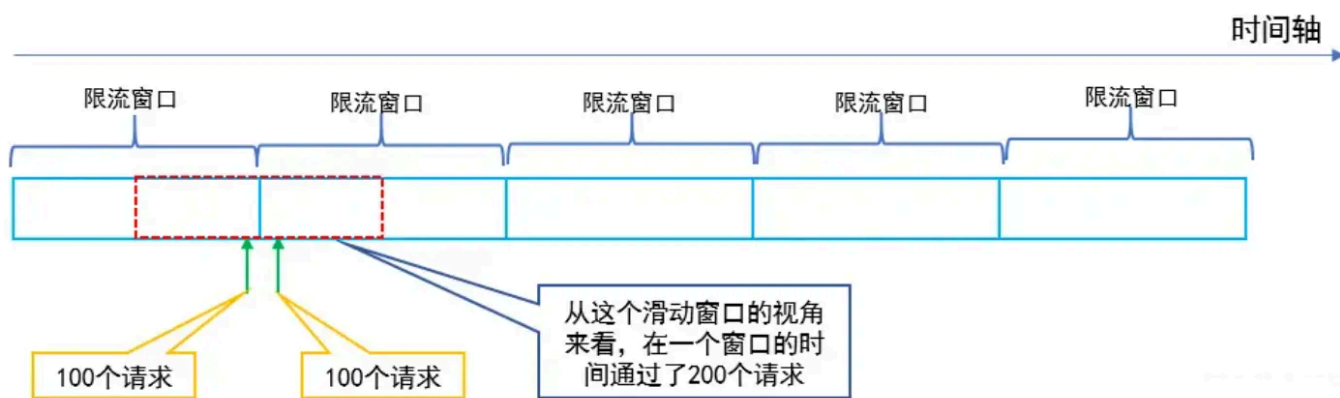
- **滑动窗口限流算法**是对固定窗口限流算法的改进，有效解决了窗口切换时可能会产生两倍于阈值流量请求的问题。
- 漏桶限流算法能够对流量起到整流的作用，让随机不稳定的流量以固定的速率流出，但是不能解决**流量突发**的问题。
- **令牌桶算法**作为漏斗算法的一种改进，除了能够起到平滑流量的作用，还允许一定程度的流量突发。

固定窗口限流算法

固定窗口限流算法就是对一段固定时间窗口内的请求进行计数，如果请求数超过了阈值，则舍弃该请求；如果没有达到设定的阈值，则接受该请求，且计数加1。当时间窗口结束时，重置计数器为0。



固定窗口限流优点是实现简单，但是会有“流量吐刺”的问题，假设窗口大小为1s，限流大小为100，然后恰好在某个窗口的第999ms来了100个请求，窗口前期没有请求，所以这100个请求都会通过。再恰好，下一个窗口的第1ms有来了100个请求，也全部通过了，那也就是在2ms之内通过了200个请求，而我们设定的阈值是100，通过的请求达到了阈值的两倍，这样可能会给系统造成巨大的负载压力。



滑动窗口限流算法

改进固定窗口缺陷的方法是采用滑动窗口限流算法，滑动窗口就是将限流窗口内部切分成一些更小的时间片，然后在时间轴上滑动，每次滑动，滑过一个时间片，就形成一个新的限流窗口，即滑动窗口。然后在这个滑动窗口内执行固定窗口算法即可。

滑动窗口可以避免固定窗口出现的放过两倍请求的问题，因为一个短时间内出现的所有请求必然在一个滑动窗口内，所以一定会被滑动窗口限流。



漏桶限流算法

漏桶限流算法是模拟水流过一个有漏洞的桶进而限流的思路，如图。



水龙头的水先流入漏桶，再通过漏桶底部的孔流出。如果流入的水量太大，底部的孔来不及流出，就会导致水桶太满溢出去。

从系统的角度来看，我们不知道什么时候会有请求来，也不知道请求会以多大的速率来，这就给系统的安全性埋下了隐患。但是如果加了一层漏斗算法限流之后，就能够保证请求以恒定的速率流出。在系统看来，请求永远是以平滑的传输速率过来，从而起到了保护系统的作用。

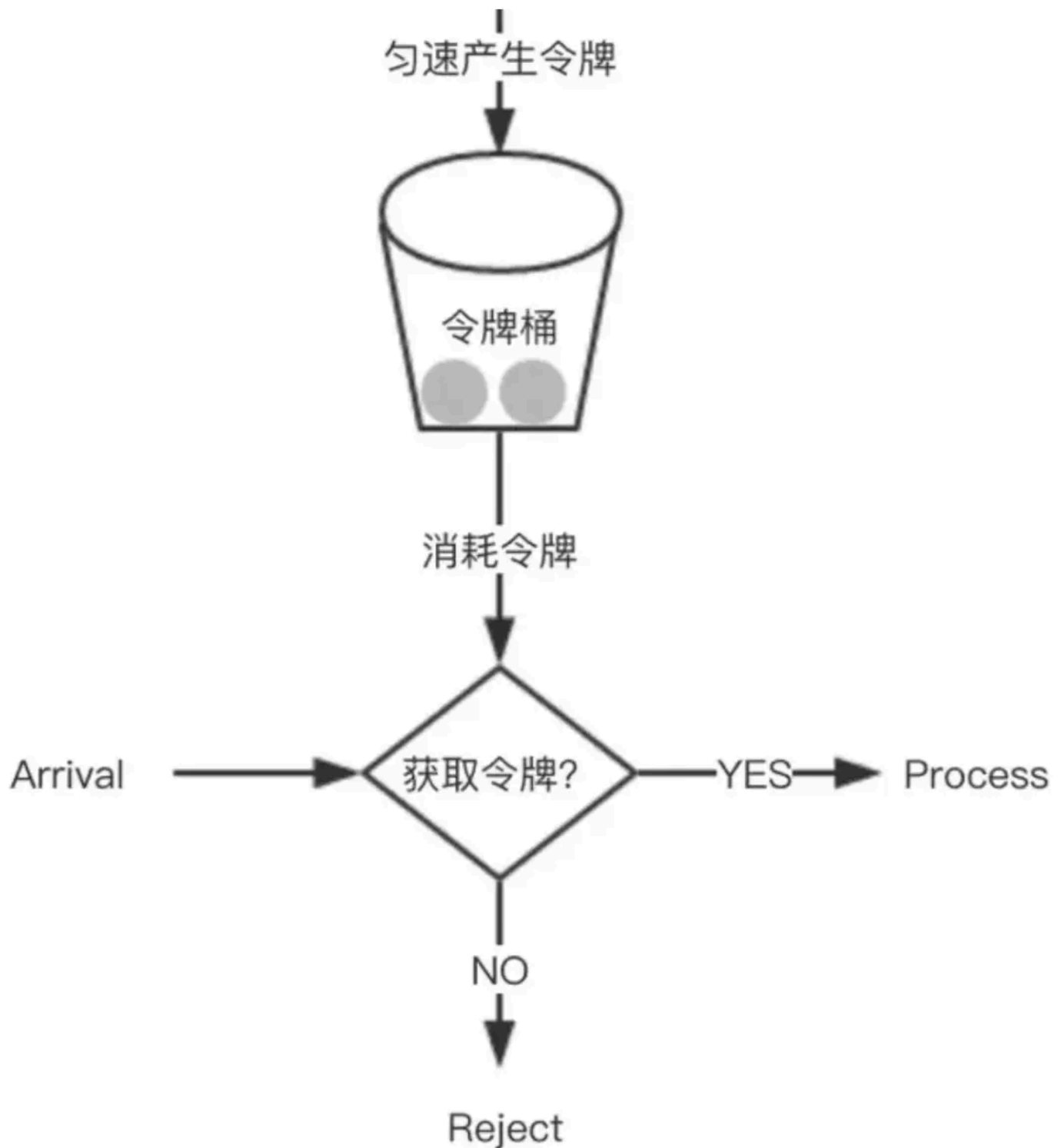
使用漏桶限流算法，缺点有两个：

- 即使系统资源很空闲，多个请求同时到达时，漏桶也是慢慢地一个接一个地去处理请求，这其实并不符合人们的期望，因为这样就是在浪费计算资源。
- 不能解决流量突发的问题，假设漏斗速率是2个/秒，然后突然来了10个请求，受限于漏斗的容量，只有5个请求被接受，另外5个被拒绝。你可能会说，漏斗速率是2个/秒，然后瞬间接受了5个请求，这不就解决了流量突发的问题吗？不，这5个请求只是被接受了，但是没有马上被处理，处理的速度仍然是我们设定的2个/秒，所以没有解决流量突发的问题

令牌桶限流算法

令牌桶是另一种桶限流算法，模拟一个特定大小的桶，然后向桶中以特定的速度放入令牌（token），请求到达后，必须从桶中取出一个令牌才能继续处理。如果桶中已经没有令牌了，那么当前请求就被限流。如果桶中的令牌放满了，令牌桶也会溢出。

放令牌的动作是持续不断进行的，如果桶中令牌数达到上限，则丢弃令牌，因此桶中可能一直持有大量的可用令牌。此时请求进来可以直接拿到令牌执行。比如设置 qps 为 100，那么限流器初始化完成 1 秒后，桶中就已经有 100 个令牌了，如果此前还没有请求过来，这时突然来了 100 个请求，该限流器可以抵挡瞬时的 100 个请求。由此可见，只有桶中没有令牌时，请求才会进行等待，最终表现的效果即为以一定的速率执行。令牌桶的示意图如下：



令牌桶限流算法综合效果比较好，能在最大程度利用系统资源处理请求的基础上，实现限流的目标，建议通常场景中优先使用该算法。

分布式一致性算法

raft协议和paxos协议的原理？

Raft 和 Paxos 是两种经典的分布式一致性算法，旨在实现**多节点状态机的高可靠一致性**。两者核心目标相同（保证分布式系统数据一致性），但设计理念和实现方式有区别。

raft 协议的原理

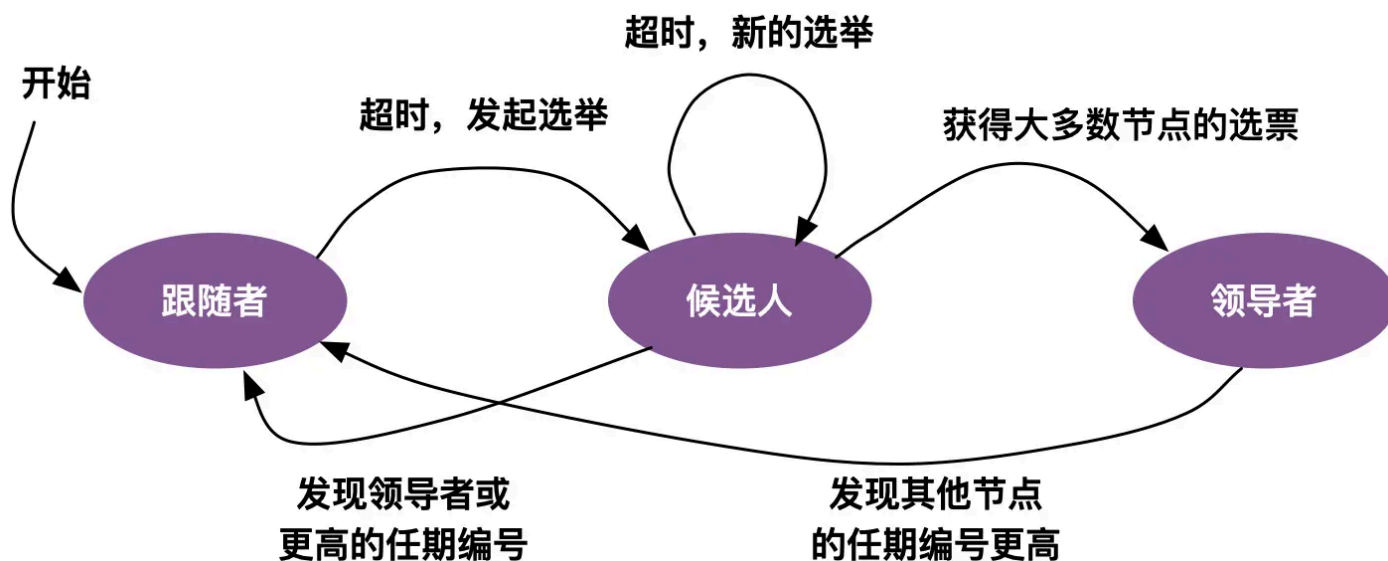
Raft算法由leader节点来处理一致性问题。leader节点接收来自客户端的请求日志数据，然后同步到集群中其它节点进行复制，当日志已经同步到超过半数以上节点的时候，leader节点再通知集群中其它节点哪些日志已经被复制成功，可以提交到raft状态机中执行。

通过以上方式，Raft算法将要解决的一致性问题的分为了以下几个子问题。

- leader选举：集群中必须存在一个leader节点。
- 日志复制：leader节点接收来自客户端的请求然后将这些请求序列化成日志数据再同步到集群中其它节点。
- 安全性：如果某个节点已经将一条提交过的数据输入raft状态机执行了，那么其它节点不可能再将相同索引的另一条日志数据输入到raft状态机中执行。

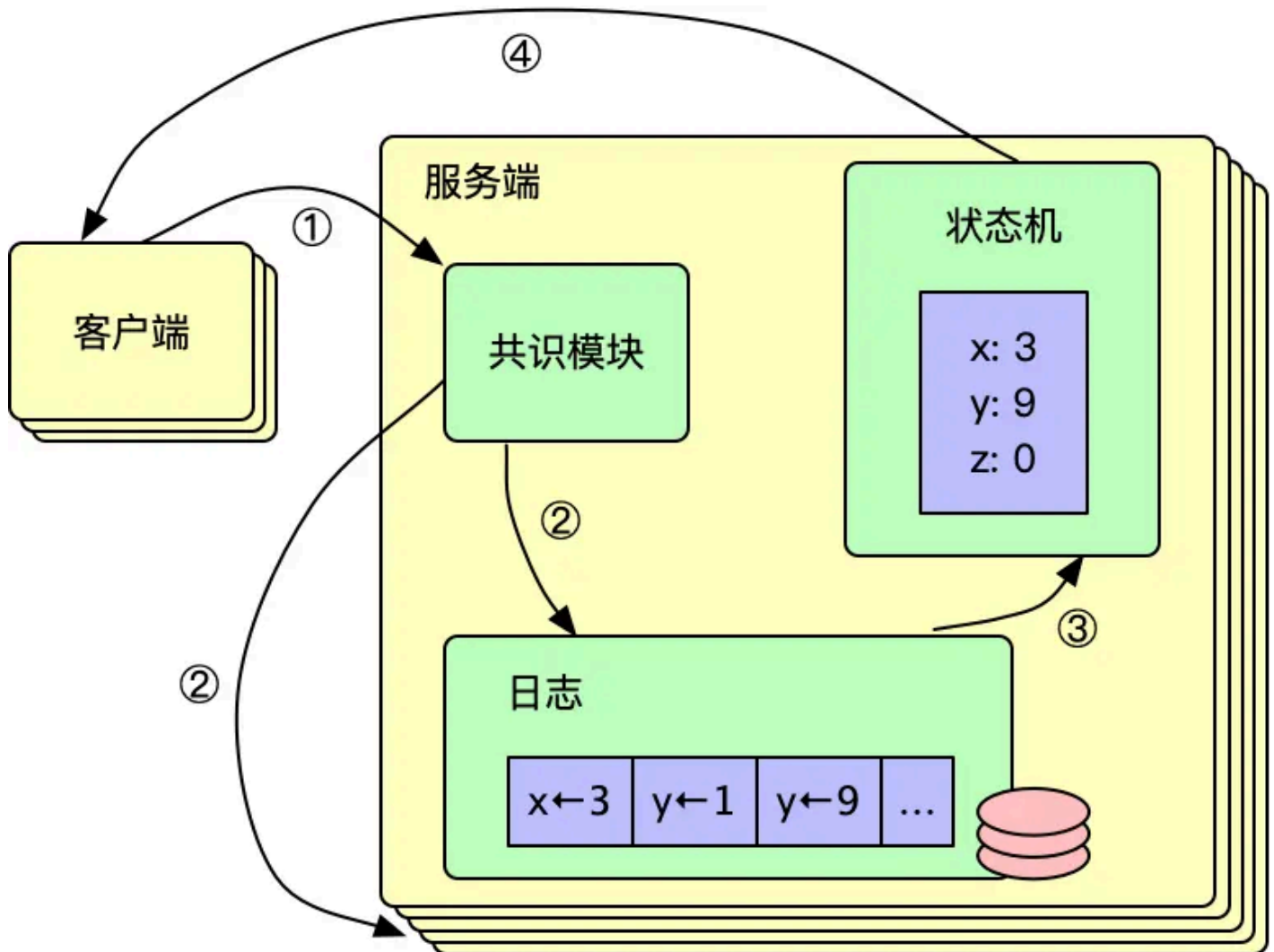
Raft算法需要有两个比较重要的机制

- **角色转换与选举机制**：Raft 将系统中的节点分为三种角色：领导者（Leader）、跟随者（Follower）和候选人（Candidate）。系统启动时，所有节点都是跟随者。跟随者会定期从领导者处接收心跳信息以确认领导者的存活。如果跟随者在一段时间内（选举超时时间）没有收到领导者的心跳，它会转变为候选人，发起新一轮的选举。候选人向其他节点发送请求投票消息。其他节点根据收到的请求投票消息，决定是否为该候选人投票。当候选人获得超过半数节点的投票时，它就成为新的领导者。领导者会周期性地向所有跟随者发送心跳消息，以维持自己的领导地位。每个领导者的领导周期称为一个任期（Term），任期号是单调递增的。



- **日志复制机制**：客户端的请求会被领导者作为日志条目添加到自己的日志中。领导者将新的日志条目复制到其他跟随者节点。它会通过附加日志消息将日志条目发送给跟随者，跟随者收到消息后会将日志条目追加到自己的日志中，并向领导者发送确认消息。当领导者得知某个日志条目已

经被大多数节点复制时，它会将该日志条目标记为已提交，并将其应用到状态机中。然后，领导者会通知其他节点该日志条目已提交，跟随者也会将已提交的日志条目应用到自己的状态机中。

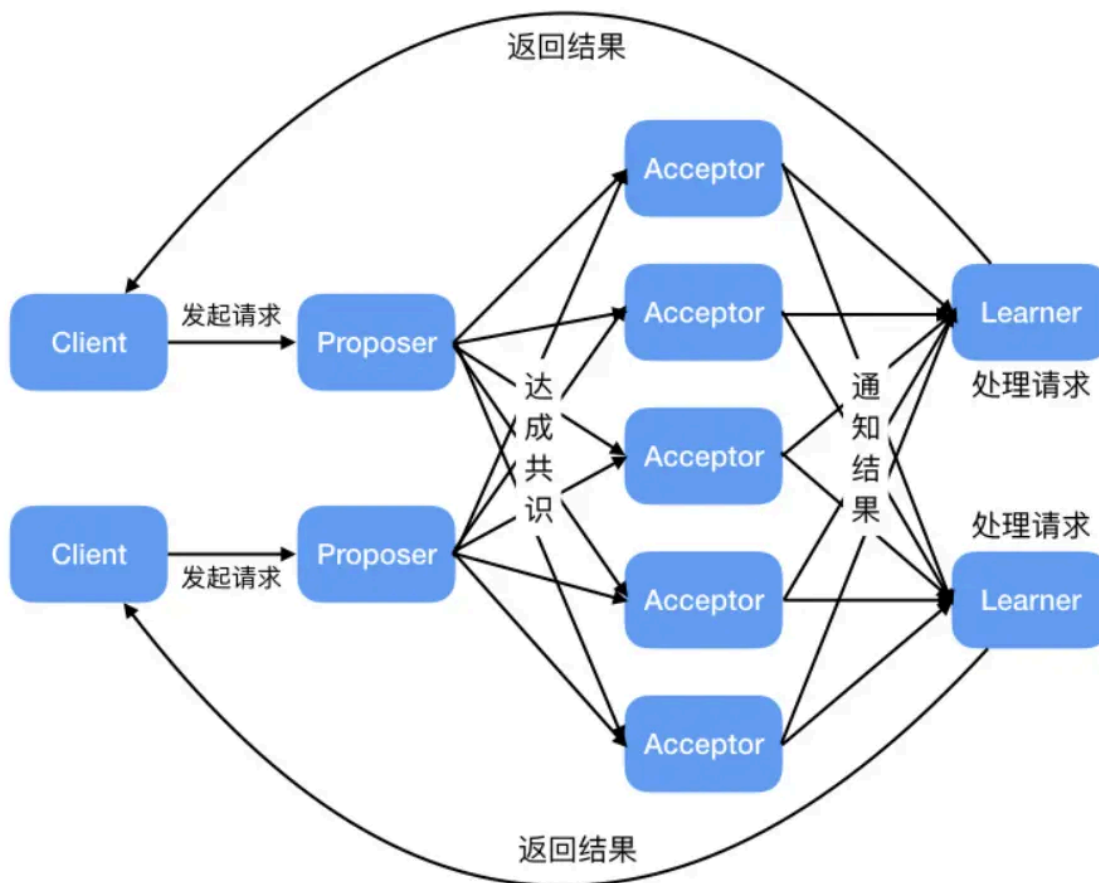


paxos协议的原理

Paxos算法的核心思想是将一致性问题分解为多个阶段，每个阶段都有一个专门的协议来处理。

Paxos算法的主要组成部分包括提议者（Proposer）、接受者（Acceptor）和投票者（Voter）。

- 提议者：提议者是负责提出一致性问题的节点，它会向接受者发送提议，并等待接受者的回复。
- 接受者：接受者是负责处理提议的节点，它会接收提议者发送的提议，并对提议进行判断。如果接受者认为提议是有效的，它会向投票者发送请求，并等待投票者的回复。
- 投票者：投票者是负责决定提议是否有效的节点，它会接收接受者发送的请求，并对请求进行判断。如果投票者认为请求是有效的，它会向接受者发送投票，表示支持或反对提议。



Paxos算法的流程如下（以Basic Paxos 算法为例子）：

- 准备阶段：提议者选择一个提案编号，并向所有接受者发送准备请求。提案编号是一个全局唯一的、单调递增的数字。接受者收到准备请求后，如果提案编号大于它之前接受过的任何提案编号，它会承诺不再接受编号小于该提案编号的提案，并返回它之前接受过的最大编号的提案信息（如果有）。
- 接受阶段：如果提议者收到了超过半数接受者的响应，它会根据这些响应确定要提议的值。如果接受者返回了之前接受过的提案信息，提议者会选择编号最大的提案中的值作为要提议的值；如果没有，提议者可以选择自己的值。提议者向所有接受者发送接受请求，包含提案编号和要提议的值。
- 学习阶段：当提议者收到超过半数接受者对某个提案的接受响应时，该提案被认为达成共识。学习者通过接受者的通知得知达成共识的值。

对比总结

- **Raft** 更易于理解和实现，它将共识过程分解为选举和日志复制两个相对独立的子问题，并且对选举超时时间等参数进行了明确的定义和限制，降低了算法的复杂度。
- **Paxos** 是一种更通用、更基础的共识算法，它的理论性更强，在学术界有广泛的研究和应用。但 Paxos 的实现相对复杂，理解和调试难度较大。

有什么框架或技术用了raft协议？

- **Etcd**: 一个开源的分布式键值存储系统，用于共享配置和服务发现等。它使用 Raft 协议来实现数据的一致性和分布式共识，确保在分布式环境中数据的可靠存储和访问，常被用于 Kubernetes 等容器编排系统中，为其提供配置管理和服务发现等功能。
- **Consul**: 是一个用于实现服务发现、配置管理和分布式一致性的工具，它使用 Raft 协议来管理集群状态和实现数据复制，确保在分布式环境中服务的注册、发现和配置信息的一致性和可靠性，为微服务架构等提供了重要的基础设施支持。